Technical Design Document

EventTracker Android Library

A pluggable, batched, retryable event-tracking SDK for Android

Document version	1.0
Status	Draft for review
Audience	Android engineers, backend, QA, PMs
Last updated	May 2026
Owner	Android Platform team

Table of Contents

1. Overview

1.1 Purpose

EventTracker is a lightweight, modular Android library for capturing analytics events from a host application and dispatching them reliably to one or more downstream destinations. Destinations include third-party analytics SDKs such as MoEngage and Firebase Analytics, as well as a first-party backend HTTP endpoint that receives batched payloads.

The library is designed to be drop-in for any Android application, framework-agnostic on the consumer side, and resilient to network failures, process death, and forced app termination.

1.2 Goals

- Provide a single, simple public API (track, identify, flush) that the host app uses regardless of how many destinations are configured.
- Support multiple analytics destinations through a uniform Adapter contract — adding a new SDK should not require changes to core dispatching logic.
- Persist every event to local storage before any network attempt, so events survive process death and OS-level kills.
- Batch events to the first-party backend to reduce battery and bandwidth cost.
- Apply bounded exponential-backoff retries with jitter, with a clear policy for what happens when retries are exhausted (dead-letter handling).
- Be observable: emit internal diagnostics so the host app and platform team can see queue depth, failure rate, and dead-letter volume.
- Be safe by default: never block the UI thread, never lose user data on a normal app upgrade, and never grow the local database without bound.

1.3 Non-goals

- Real-time streaming. Events are batched and may be delivered seconds to minutes after they are tracked.
- On-device aggregation, funnels, or experimentation logic.
- Cross-app or cross-device event reconciliation.
- Replacing the destination SDKs themselves — adapters wrap them, they do not reimplement them.

1.4 Glossary

Term	Definition
Event	A single user or system action with a name and a map of properties.
Adapter	A class that knows how to deliver events to one specific destination (e.g. MoEngage, Firebase, Backend).
Destination	The system that ultimately consumes events. May be an SDK or an HTTP endpoint.
Batch	A collection of events sent together in one network request to the backend destination.
DLQ	Dead-letter queue — storage for events that failed delivery after all retries were exhausted.
Flush	An explicit instruction to send everything currently queued, regardless of normal batch triggers.
Backoff	Increasing delay between retry attempts to avoid overloading a struggling backend.

2. Requirements

2.1 Functional requirements

ID	Requirement
FR-1	Host app can track an event with a name and an arbitrary properties map.
FR-2	Host app can identify a user with a stable user ID and traits map.
FR-3	Host app can configure one or more adapters at SDK initialization time.
FR-4	All events are persisted to a local database before any network or SDK delivery attempt.
FR-5	Events destined for the backend are batched by size or by elapsed time, whichever comes first.
FR-6	Failed backend deliveries are retried with exponential backoff and jitter, up to a configured maximum.
FR-7	Events that exhaust all retries are moved to a dead-letter queue and not deleted automatically.
FR-8	Host app can manually flush, inspect dead-letter size, and purge the dead-letter queue.
FR-9	Library performs all I/O off the main thread.
FR-10	Library survives process death and resumes pending deliveries on next app start or scheduled job run.
FR-11	Each adapter can opt in or out per-event using a tag/destination filter, so PII or sensitive events can be restricted.
FR-12	Library exposes diagnostic counters (queued, sent, failed, dead-lettered) for observability.

2.2 Non-functional requirements

ID	Requirement	Target
NFR-1	Track call latency on caller thread	< 1 ms p95 (enqueue is fire-and-forget to a worker)
NFR-2	Local DB size cap	Configurable, default 10,000 events; oldest events drop with telemetry on overflow
NFR-3	Battery impact	No wake-locks, no foreground services, scheduled work via WorkManager
NFR-4	Min Android version	API 21 (Lollipop) and above
NFR-5	APK size impact	< 250 KB AAR for core + one adapter
NFR-6	Thread safety	Public API safe to call from any thread
NFR-7	Crash safety	Library exceptions never propagate to host app
NFR-8	Privacy	No automatic PII collection; payload is exactly what host app provides

2.3 Crucial requirements added during design

The following requirements were not in the original brief but are essential and have been added:

- **Schema versioning.** Each persisted event carries a schemaVersion so payload format changes can be migrated without data loss.
- **App-lifecycle awareness.** On app foreground/background and on network connectivity changes, the dispatcher re-evaluates whether to flush.
- **Opt-out / GDPR support.** A single setOptOut(true) call must stop all collection and offer to wipe local storage.
- **Idempotency.** Each event has a UUID; the backend uses it to deduplicate retried batches.
- **Clock-skew handling.** Events store both clientTimestamp (device clock) and clientUptimeMs (monotonic), so the backend can correct skew.
- **Configurable encryption-at-rest.** Optional SQLCipher/EncryptedFile wrapping for sensitive deployments.
- **Sampling and rate limits.** Per-event-name sampling rate so verbose events (e.g. scroll) can be down-sampled at source.
- **Backend protocol contract.** Defined batch envelope, response codes, and Retry-After honoring — see Section 6.4.
- **Process-isolation safety.** If host app uses multiple processes, the SDK must initialize only once per process and serialize DB access via a single WAL-mode SQLite handle.

3. High-Level Architecture

The library is organized into five layers, each with a single responsibility. Data flows downward from the public API into persistence, then outward to one or more adapters. Diagnostics flow back upward.

3.1 Component diagram

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ Host Application │
│ │
│ EventTracker.track("checkout_started", mapOf("cart" to 5)) │
└──────────────────────────────────────────────────────────────────────────────┘
                                │
                                ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│ 1. Public API layer │
│ EventTracker · EventTrackerConfig · TrackEvent │
└──────────────────────────────────────────────────────────────────────────────┘
                                │
                                ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│ 2. Core layer │
│ EventDispatcher · SamplingFilter · OptOutGuard · SessionMgr │
```

```
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│ 3. Persistence layer                                         │
│ EventDao · EventEntity · EventDatabase (Room) · DLQDao       │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│ 4. Adapter layer                                             │
│ EventAdapter (interface)                                     │
│  ├── BackendBatchAdapter (uses HTTP + RetryPolicy)           │
│  ├── MoEngageAdapter                                         │
│  ├── FirebaseAdapter                                         │
│  └── <vendor-X>Adapter                                       │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│ 5. Scheduling & networking                                   │
│ FlushScheduler (WorkManager) · HttpClient · RetryPolicy      │
└──────────────────────────────────────────────────────────────┘
```

3.2 Data flow for a tracked event

1. Host calls EventTracker.track(name, props). The call is non-blocking.
2. OptOutGuard checks the opt-out flag; if opted out, the event is dropped.
3. SamplingFilter evaluates the per-event sampling rate; the event may be dropped here.
4. EventDispatcher constructs a TrackEvent with a UUID, schema version, and timestamps, then enqueues it on a single-threaded coroutine dispatcher.
5. EventDispatcher writes the event to the local Room database (events table).
6. EventDispatcher fans out the event to each configured adapter that accepts it (per the destination filter).
7. Realtime adapters (MoEngage, Firebase) deliver immediately; the BackendBatchAdapter only marks the row as queued.
8. FlushScheduler triggers a flush when batch size, batch age, or app lifecycle conditions are met.

9. On flush, BackendBatchAdapter reads up to N queued rows, sends them as one HTTP POST, and on success deletes the rows.

10. On failure, the rows have their attemptCount incremented and are scheduled for retry per RetryPolicy. After max retries, rows are moved to the dead-letter table.

3.3 Threading model

Component	Thread / Dispatcher
Public API (track, identify)	Caller thread; immediately hands off to internal coroutine
EventDispatcher internal queue	Single-threaded CoroutineDispatcher (Executors.newSingleThreadExecutor().asCoroutineDispatcher())
Room I/O	Dispatchers.IO (Room handles its own confinement)
Realtime adapter delivery	Dispatchers.Default (CPU-light)
Batch HTTP send	WorkManager-provided thread
Adapter callbacks	Always on EventDispatcher's single thread (so adapters do not need to be reentrant)

Why single-threaded?

A single internal dispatcher gives us strict ordering, no read-modify-write races on the database, and adapters that can be written without locks. Throughput is more than sufficient because tracking is bursty and the actual HTTP send happens on a separate WorkManager thread.

4. Public API

The public API surface is intentionally small. The host app only needs to know about EventTracker, EventTrackerConfig, the adapter interface (if writing a new adapter), and a handful of supporting types.

4.1 EventTracker (entry point)

Singleton facade exposed to the host application. All public calls are thread-safe and non-blocking.

KDoc

```
/**
 * EventTracker is the single entry point for the EventTracker Android library.
 *
 * The library captures events in the host app, persists them to a local
 * database, and dispatches them to one or more configured [EventAdapter]
 * destinations. Network destinations use batched delivery with retry and
 * dead-letter handling. See the package-level documentation for the full
 * data flow.
 *
 * Initialization must happen exactly once per process, typically in
 * [Application.onCreate]. After initialization, all calls are safe from
 * any thread.
```

```
*
* Example:
* ```
* EventTracker.initialize(
* context = applicationContext,
* config = EventTrackerConfig.Builder()
*  .addAdapter(BackendBatchAdapter(endpoint = "https://api.example.com/events"))
*  .addAdapter(FirebaseAdapter())
*  .addAdapter(MoEngageAdapter(appId = "XYZ"))
*  .batchSize(50)
*  .batchIntervalMs(30_000)
*  .maxRetries(8)
*  .build()
* )
*
* EventTracker.track("checkout_started", mapOf("cart_size" to 3))
* ```
*
* @see EventTrackerConfig
* @see EventAdapter
* @since 1.0.0
*/
object EventTracker {
/**
* Initialize the library. Safe to call multiple times; subsequent calls
* are no-ops and log a warning. Must be called before any [track] or
* [identify] call.
*
* @param context Application context. The library retains only the
* application context, never an activity context.
* @param config Immutable configuration produced by
```

```
* [EventTrackerConfig.Builder].
* @throws IllegalArgumentException if config has zero adapters.
*/
fun initialize(context: Context, config: EventTrackerConfig)
/**
* Track an event. Returns immediately; persistence and delivery happen
* on a background dispatcher.
*
* @param name Event name. Must be 1..128 chars, alphanumeric +
* underscore. Names outside this pattern are rejected and logged.
* @param properties Event properties. Values must be JSON-serializable
* primitives, lists, or nested maps. Non-serializable values are
* coerced to their toString() representation and a diagnostic counter
* is incremented.
* @param destinations Optional whitelist of destination IDs (e.g.
* "backend", "firebase"). When null, all configured adapters
* that accept the event will receive it.
*/
fun track(
name: String,
properties: Map<String, Any?> = emptyMap(),
destinations: Set<String>? = null,
)
/**
* Associate a stable user identifier and optional user traits with the
* current session. Subsequent events will carry this user id until
* [reset] is called.
*
* @param userId Stable user identifier. Pass `null` to clear.
* @param traits Optional user attributes (email, plan, etc.).
*/
```

```
fun identify(userId: String?, traits: Map<String, Any?> = emptyMap())

/**

* Forget the current user and start a new anonymous session. Does not

* clear queued events; those continue to be delivered with the user id

* they were tracked under.

*/

fun reset()

/**

* Force an immediate flush of all queued backend events, bypassing

* batch-size and batch-interval triggers. The returned Job completes

* when the flush attempt finishes (success or failure).

*/

fun flush(): Job

/**

* Set the opt-out flag. When true, all subsequent [track] and [identify]

* calls are dropped silently. Already-persisted events are NOT deleted

* unless [wipeLocalData] is also called.

*/

fun setOptOut(optedOut: Boolean)

/**

* Delete all locally persisted events, including the dead-letter

* queue. Intended for GDPR / opt-out compliance and for QA reset.

* Does not affect events already delivered to destinations.

*/

fun wipeLocalData(): Job

/**

* Snapshot of internal counters for observability.

*/

fun diagnostics(): Diagnostics

}
```

4.2 EventTrackerConfig

Immutable configuration object built via a fluent builder.

```
/**
* Immutable configuration for [EventTracker]. Use [Builder] to construct.
*
* @property adapters Ordered list of destinations. Order matters only for
* logging; runtime delivery is parallelized where safe.
* @property batchSize Maximum number of events in a single backend batch.
* Range: 1..1000. Default: 50.
* @property batchIntervalMs Maximum time, in milliseconds, an event may sit
* in the queue before forcing a flush. Default: 30_000 (30 s).
* @property maxRetries Maximum number of HTTP retry attempts for a batch
* before its events are moved to the dead-letter queue. Default: 8.
* @property maxLocalEvents Hard cap on the events table. When exceeded,
* oldest non-dead-letter rows are dropped. Default: 10_000.
* @property samplingRates Per-event-name sampling rates in 0.0..1.0.
* Names not present default to 1.0 (always tracked).
* @property encryptAtRest If true, the database is opened with SQLCipher
* using a key from the Android Keystore. Default: false.
* @property logger SLF4J-style logger for internal diagnostics.
* Default: NoOpLogger.
*/
class EventTrackerConfig private constructor(
val adapters: List<EventAdapter>,
val batchSize: Int,
val batchIntervalMs: Long,
val maxRetries: Int,
val maxLocalEvents: Int,
val samplingRates: Map<String, Double>,
val encryptAtRest: Boolean,
```

```
val logger: EventLogger,
) {
class Builder {
fun addAdapter(adapter: EventAdapter): Builder
fun batchSize(size: Int): Builder
fun batchIntervalMs(ms: Long): Builder
fun maxRetries(n: Int): Builder
fun maxLocalEvents(n: Int): Builder
fun samplingRate(eventName: String, rate: Double): Builder
fun encryptAtRest(enabled: Boolean): Builder
fun logger(logger: EventLogger): Builder
/** @throws IllegalStateException if no adapters have been added. */
fun build(): EventTrackerConfig
}
}
```

4.3 TrackEvent (internal model)

The canonical in-memory representation of an event. Used by all internal layers and adapters.

```
/**
* A single event flowing through the pipeline.
*
* @property id Locally unique UUID generated at track-time. Used as the
* idempotency key on the backend.
* @property name Validated event name.
* @property properties Event payload, after sanitization.
* @property userId Identifier set by [EventTracker.identify], if any.
* @property sessionId Identifier of the session this event belongs to.
* @property clientTimestamp Wall-clock time of the track call (UTC ms).
* @property clientUptimeMs Monotonic uptime at track-time, used to correct
* wall-clock skew on the backend.
* @property schemaVersion Format version. The current code emits 1.
```

```
* @property destinations Whitelist passed by the caller, or null for "all".
* @property attemptCount Number of delivery attempts so far. Zero on creation.
*/
data class TrackEvent(
val id: String,
val name: String,
val properties: Map<String, Any?>,
val userId: String?,
val sessionId: String,
val clientTimestamp: Long,
val clientUptimeMs: Long,
val schemaVersion: Int = 1,
val destinations: Set<String>? = null,
val attemptCount: Int = 0,
)
```

4.4 Diagnostics

```
/**
* Read-only snapshot of internal counters. Values are monotonically
* increasing within a process lifetime and reset to zero on process start.
*
* @property tracked Total events accepted by [EventTracker.track].
* @property dropped Events rejected by validation, sampling, or opt-out.
* @property persisted Events written to the local database.
* @property delivered Events successfully delivered to all required adapters.
* @property retrying Events currently waiting for a retry attempt.
* @property deadLettered Events moved to the dead-letter queue.
* @property queueDepth Current number of un-delivered rows in events table.
*/
data class Diagnostics(
val tracked: Long,
```

```
val dropped: Long,

val persisted: Long,

val delivered: Long,

val retrying: Long,

val deadLettered: Long,

val queueDepth: Long,

)
```

5. Adapter Layer

Adapters are the only place that knows about a specific destination. The Adapter contract is small and uniform, which keeps integrations independent of the dispatcher and of one another.

5.1 EventAdapter contract

```
/**

* Contract for a destination that receives events. Implementations may

* deliver immediately (in-process SDKs) or batch over the network.

*

* Implementations must:

* - Be safe to invoke from a single background thread (the dispatcher

* serializes calls per adapter instance).

* - Never throw to the caller. Wrap exceptions and return a [Result] /

* failure outcome instead.

* - Be idempotent: the same event may be delivered more than once if the

* library cannot distinguish a successful delivery (e.g. timeout after

* server received the request).

*

* @since 1.0.0

*/

interface EventAdapter {

/** Stable identifier (e.g. "firebase", "moengage", "backend"). */

val id: String

/** Called once during [EventTracker.initialize]. */

fun initialize(context: Context, logger: EventLogger)
```

```
/**
* Decide whether this adapter wants the given event. Allows adapters
* to filter by event name, by destination whitelist, or by config.
*/
fun accepts(event: TrackEvent): Boolean
/**
* Deliver a single event. Realtime adapters implement this; batch
* adapters typically just enqueue.
*
* @return [DeliveryOutcome.Success] on success,
* [DeliveryOutcome.RetryableFailure] for transient errors (the
* dispatcher will retry on its own schedule),
* [DeliveryOutcome.PermanentFailure] for non-retryable errors
* (the event is moved straight to the DLQ).
*/
suspend fun deliver(event: TrackEvent): DeliveryOutcome
/**
* Forwarded user identification. Default implementation is a no-op.
*/
suspend fun identify(userId: String?, traits: Map<String, Any?>) {}
/** Called from [EventTracker.flush]. Default no-op. */
suspend fun flush(): DeliveryOutcome = DeliveryOutcome.Success
/** Called when the host opts out and clears local data. */
fun onOptOut() {}
}
sealed class DeliveryOutcome {
object Success : DeliveryOutcome()
data class RetryableFailure(val cause: Throwable, val retryAfterMs: Long? = null) : DeliveryOutcome()
data class PermanentFailure(val cause: Throwable) : DeliveryOutcome()
}
```

5.2 Built-in adapters

Adapter	id	Behavior
BackendBatchAdapter	backend	Persists rows as 'queued', batched HTTP POST on flush, retry with backoff, DLQ on exhaustion.
FirebaseAdapter	firebase	Wraps FirebaseAnalytics.logEvent. Delivers immediately. Maps property types per Firebase rules (Bundle).
MoEngageAdapter	moengage	Wraps MoEngage's MoEAnalyticsHelper.trackEvent. Delivers immediately. Translates Map to Properties.
LoggingAdapter	log	Debug helper; logcat only. accepts() returns true only in debug builds.

5.3 Writing a custom adapter

To add a new destination, implement EventAdapter and register it in the config builder. Below is a minimal example for a hypothetical 'Mixpanel' adapter.

```
class MixpanelAdapter(private val token: String) : EventAdapter {

override val id: String = "mixpanel"

private lateinit var mp: MixpanelAPI

override fun initialize(context: Context, logger: EventLogger) {

mp = MixpanelAPI.getInstance(context, token)

}

override fun accepts(event: TrackEvent): Boolean =

event.destinations?.contains(id) ?: true

override suspend fun deliver(event: TrackEvent): DeliveryOutcome = try {

val props = JSONObject(event.properties)

mp.track(event.name, props)

DeliveryOutcome.Success

} catch (t: Throwable) {

DeliveryOutcome.RetryableFailure(t)

}

override suspend fun identify(userId: String?, traits: Map<String, Any?>) {

userId?.let { mp.identify(it) }

mp.people.set(JSONObject(traits))

}

}
```

5.4 Sequence: realtime vs batched delivery

```
Host Tracker DB FirebaseAdapter BackendBatchAdapter Network
│ track(e) │ │ │ │ │
│──────────────────▶│ │ │ │ │
│ │ insert(e) │ │ │ │
│ │──────────────────▶│ │ │ │
│ │ │ │ │ │
│ │ deliver(e) │ │ │
│ │──────────────────────────────────────▶│ │ │
│ │ │ │ (logEvent, sync) │ │
│ │ deliver(e) │ │
│ │──────────────────────────────────────────────────────────────▶│ │
│ │ │ │ markQueued(e) │
│ │ │◀──────────────────────────────────────────────────────│ │
│ (later) │ │
│ FlushScheduler ──────────────── trigger flush ──────────────────────────────▶│ │
│ readBatch(N) │ │
│ ◀──────────────────────────────────────────────────────│ │
│ │ POST /events │
│ │──────────────────────▶│
│ │ 200 OK │
│ │◀──────────────────────│
│ deleteRows(ids) │ │
│ ◀──────────────────────────────────────────────────────│ │
```

6. Persistence and Batched Backend Delivery

6.1 Storage: Room schema

Two tables: events for the live queue, and dead_letter_events for terminal failures. Both are managed by the same Room database.

```
@Entity(
tableName = "events",
indices = [
```

```
Index(value = ["state", "next_attempt_at"]),
Index(value = ["created_at"])
]
)
data class EventEntity(
@PrimaryKey val id: String, // UUID
@ColumnInfo(name = "name") val name: String,
@ColumnInfo(name = "payload_json") val payloadJson: String,
@ColumnInfo(name = "user_id") val userId: String?,
@ColumnInfo(name = "session_id") val sessionId: String,
@ColumnInfo(name = "client_ts") val clientTs: Long,
@ColumnInfo(name = "client_uptime_ms") val clientUptimeMs: Long,
@ColumnInfo(name = "schema_version") val schemaVersion: Int,
@ColumnInfo(name = "destinations_csv") val destinationsCsv: String?,
@ColumnInfo(name = "state") val state: String, // QUEUED, SENDING, FAILED
@ColumnInfo(name = "attempt_count") val attemptCount: Int,
@ColumnInfo(name = "next_attempt_at") val nextAttemptAt: Long,
@ColumnInfo(name = "last_error") val lastError: String?,
@ColumnInfo(name = "created_at") val createdAt: Long,
)
@Dao
interface EventDao {
@Insert(onConflict = OnConflictStrategy.REPLACE)
suspend fun insert(event: EventEntity)
@Query("""
SELECT * FROM events
WHERE state = 'QUEUED' AND next_attempt_at <= :now
ORDER BY created_at ASC
LIMIT :limit
""")
suspend fun nextBatch(now: Long, limit: Int): List<EventEntity>
```

```
@Query("UPDATE events SET state = 'SENDING' WHERE id IN (:ids)")

suspend fun markSending(ids: List<String>)

@Query("DELETE FROM events WHERE id IN (:ids)")

suspend fun deleteSucceeded(ids: List<String>)

@Query("""

UPDATE events

SET state = 'QUEUED',

attempt_count = attempt_count + 1,

next_attempt_at = :nextAt,

last_error = :err

WHERE id IN (:ids)

""")

suspend fun rescheduleFailed(ids: List<String>, nextAt: Long, err: String)

@Query("SELECT COUNT(*) FROM events")

suspend fun queueDepth(): Long

@Query("DELETE FROM events WHERE id IN (SELECT id FROM events ORDER BY created_at ASC LIMIT
:n)")

suspend fun trimOldest(n: Int)

}
```

6.2 Batching policy

A flush is triggered by the first of three conditions to be met:

- Size trigger: queue depth >= batchSize.
- Time trigger: oldest queued event is older than batchIntervalMs.
- External trigger: app foregrounded, network became available, or host called flush().

WorkManager schedules a periodic worker (default every batchIntervalMs / 2, with constraints CONNECTED and BATTERY_NOT_LOW) and a one-shot worker on external triggers. Constraints are intentionally lenient: the SDK does not require unmetered networks, because that surprises users in metered-network markets.

6.3 Backend wire format

All batches are sent as a single POST to {endpoint}. Content-Type is application/json; charset=utf-8. Compression is gzip when payload exceeds 1 KiB.

```
POST /v1/events HTTP/1.1

Host: api.example.com
```

```
Authorization: Bearer <client-token>

Content-Type: application/json; charset=utf-8

Content-Encoding: gzip

X-Eventtracker-Sdk: android/1.0.0

X-Eventtracker-Batch-Id: 8c8c1e3a-... # UUID per batch attempt

X-Eventtracker-Idempotency-Key: <batch-id> # used by backend to dedupe

{

"schema_version": 1,

"sent_at": 1714650000000,

"device": {

"os": "android",

"os_version": "14",

"app_version": "3.2.1",

"model": "Pixel 8",

"locale": "en-IN"

},

"events": [

{

"id": "8c8c1e3a-1b54-4c52-9b7e-...", # idempotency key per event

"name": "checkout_started",

"properties": { "cart_size": 3 },

"user_id": "u_42",

"session_id": "s_19",

"client_ts": 1714649999987,

"client_uptime_ms": 4823145,

"attempt_count": 0

}

// ... up to batchSize entries

]

}
```

6.4 Backend response contract

HTTP status	Meaning	SDK behavior
200 / 202	Whole batch accepted	Delete all rows in the batch from local DB.
207	Per-event partial success (body lists failed event ids)	Delete successes; reschedule listed failures with backoff.
400	Validation error (bad schema, oversized payload)	Move all events in the batch to DLQ. Do not retry — the payload will fail again.
401 / 403	Auth failure	Stop attempts, surface AuthError via diagnostics, retry next app start with fresh token.
408 / 429 / 5xx	Transient failure	Reschedule with backoff. Honor Retry-After header if present.
Network exception (no response)	Transient failure	Reschedule with backoff. Increment attempt_count.

6.5 Retry policy

Retries use truncated exponential backoff with full jitter. Given baseDelayMs and maxDelayMs:

```
delay(n) = random(0, min(maxDelayMs, baseDelayMs * 2^n))

Defaults: baseDelayMs = 1_000 // 1 s

maxDelayMs = 600_000 // 10 min

maxRetries = 8

Worst-case wait per event = sum over n=0..7 of up to delay(n)

≈ 1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 s (capped at 600 s)

≈ 4 minutes 15 seconds nominal,

up to ~80 minutes with capped retries on a sustained outage.
```

Full jitter (random 0..delay) is preferred over equal jitter because it spreads thundering-herd waves more evenly when many devices retry against a recovering backend.

If the server returns a Retry-After header, the SDK uses max(serverHint, computedDelay) so the server can lengthen but never shorten the wait.

6.6 What happens after retries are exhausted (DLQ)

When attempt_count reaches maxRetries on a 4xx-non-auth or repeated 5xx failure, the affected rows are moved atomically from events to dead_letter_events in the same transaction. They are not deleted automatically because:

- They may indicate a real backend bug worth investigating after the fact.
- If the host app exposes them, the engineering team can re-attempt later (e.g. after a server fix) by calling EventTracker.replayDeadLetters(limit).

The DLQ is bounded: when it exceeds maxLocalEvents (or a separate maxDeadLetters cap, default 5,000), oldest entries are dropped and a deadLetterDropped diagnostic is incremented.

DLQ table

```
@Entity(tableName = "dead_letter_events")
```

```
data class DeadLetterEntity(
@PrimaryKey val id: String,
val name: String,
val payloadJson: String,
val attemptCount: Int,
val firstFailureAt: Long,
val lastFailureAt: Long,
val lastError: String,
val httpStatus: Int?,
val schemaVersion: Int,
val createdAt: Long,
)
```

DLQ public API

```
/**
* Number of events currently in the dead-letter queue.
*/
suspend fun deadLetterSize(): Long
/**
* Move up to [limit] DLQ rows back into the live queue, with attempt_count
* reset to zero. Useful after a backend fix.
*
* @return the number of events actually re-queued.
*/
suspend fun replayDeadLetters(limit: Int = 500): Int
/**
* Permanently delete all DLQ entries.
*/
suspend fun purgeDeadLetters()
```

6.7 Failure-handling decision matrix

Failure source	Classification	Outcome
No network	Retryable	Backoff retry; flush on connectivity-available broadcast.
Server 5xx	Retryable	Backoff retry, honor Retry-After.
Server 429	Retryable + throttle	Honor Retry-After; halve batch size for next attempt.
Server 400	Permanent (payload)	Move to DLQ immediately; do not retry.
Server 401/403	Auth	Halt sender, raise diagnostic, retry next process start.
DB write error	Local-fatal	Drop event, increment dropped counter, log.
Adapter exception (realtime)	Adapter-local	Caught, logged; never affects other adapters or DB row.
maxRetries reached	Terminal	Move row to DLQ table in same transaction as removal from events.

7. Lifecycle, Concurrency, Edge Cases

7.1 App lifecycle hooks

Event	Action
ProcessLifecycleOwner ON_START	Schedule a one-shot flush worker (constraint: CONNECTED).
ProcessLifecycleOwner ON_STOP	Persist a sessionEnd marker; do not force flush (avoid background work right when user leaves).
Connectivity becomes available	Enqueue one-shot flush worker.
WorkManager periodic worker	Runs every batchIntervalMs/2; flushes if queue has work and constraints satisfied.
Forced app stop / process death	Nothing — events on disk are picked up by the next periodic worker run or next app start.

7.2 Multi-process safety

If the host app uses multiple Android processes (e.g. a separate :pushservice process), each process will load the SDK separately. To avoid SQLite contention:

- Database is opened in WAL mode (default in Room).
- Only one process — the main process — runs the FlushScheduler. The SDK detects this via android:process attribute and disables the scheduler in non-main processes.
- Non-main processes still write to the same DB; their events are picked up by the main process's scheduler.

7.3 App-version migrations

Schema migrations follow Room's standard Migration mechanism. Events with an older schemaVersion than the current code are processed through a SchemaUpgrader before being sent. Rows with a schema version newer than the running code (e.g. user downgraded) are left in place and a downgradeFound diagnostic is incremented.

7.4 Clock-skew and reordering

Wall-clock time on Android can jump (NTP, manual change, daylight savings). Each event therefore stores both client_ts (wall-clock UTC) and client_uptime_ms (SystemClock.elapsedRealtime). The backend can detect skew by comparing two events from the same session and rebase them if needed. The SDK never relies on client_ts for ordering — local ordering is always by created_at insertion column.

7.5 Privacy and opt-out

- setOptOut(true) sets a SharedPreferences flag and short-circuits track / identify.

- setOptOut(true) does not delete data unless wipeLocalData() is also invoked. This separation is deliberate: some compliance regimes require retaining already-collected data for audit.
- On opt-in re-enabling, no historical data is recovered — events dropped during opt-out are gone for good.

7.6 Crash safety

All public-API entry points are wrapped in a try/catch that logs and increments a sdkException counter. A bug in the SDK must never crash the host app.

8. Observability and Testing

8.1 Internal logging

All internal logs flow through an EventLogger interface. Default implementation is a no-op so production builds are silent unless the host explicitly opts in. Log levels: VERBOSE, DEBUG, INFO, WARN, ERROR. Tags are namespaced as ET/<component>, e.g. ET/Dispatcher, ET/Backend, ET/Retry.

8.2 Diagnostic counters

Already shown in Section 4.4. Hosts may expose them in a debug screen or forward them to their APM (Datadog, New Relic).

8.3 Test strategy

Layer	Test type	Notes
Public API	Unit (JVM)	Verify validation, opt-out short-circuit, sampling.
EventDispatcher	Unit (JVM, coroutine test)	Inject fake adapters and DB.
Room schema	Instrumented (Robolectric or Android Test)	Migration tests for every schema change.
BackendBatchAdapter	MockWebServer	Cover 200, 207, 4xx, 5xx, 429 with Retry-After, network drop.
RetryPolicy	Unit	Property tests on backoff bounds and jitter distribution.
End-to-end	Espresso + MockWebServer	Track events, kill app, verify resumed delivery.

8.4 Acceptance criteria

- 100,000 events tracked over a 24-hour soak test result in zero loss to either delivery or DLQ.
- With the backend offline for 30 minutes, all events queued during the outage are eventually delivered after recovery.
- With the backend returning 400 for a poison batch, the affected events end up in the DLQ within maxRetries-equivalent time and the live queue continues draining.
- Process kill at any point during a track or flush leaves the database in a consistent state.

9. Documentation Generation (Dokka)

All public types and functions in the library carry KDoc as illustrated in Sections 4 and 5. The build emits HTML and Markdown documentation via Dokka, which is wired into the Gradle build.

9.1 Gradle setup

```
plugins {

id("com.android.library")
```

```
id("org.jetbrains.kotlin.android")
id("org.jetbrains.dokka") version "1.9.20"
}
dokka {
dokkaPublications.html {
outputDirectory.set(layout.buildDirectory.dir("docs/html"))
}
dokkaPublications.gfm {
outputDirectory.set(layout.buildDirectory.dir("docs/markdown"))
}
dokkaSourceSets.named("main") {
moduleName.set("eventtracker")
includes.from("module.md")
sourceLink {
localDirectory.set(file("src/main/kotlin"))
remoteUrl("https://github.com/example/eventtracker/tree/main/src/main/kotlin")
remoteLineSuffix.set("#L")
}
externalDocumentationLinks.register("androidx") {
url("https://developer.android.com/reference/")
}
perPackageOption {
matchingRegex.set("com\\.example\\.eventtracker\\.internal.*")
suppress.set(true) // hide internal package from public docs
}
}
}
tasks.register("publishDocs") {
dependsOn("dokkaHtml", "dokkaGfm")
}
```

9.2 KDoc conventions

- First sentence is a single-line summary that stands on its own.
- Use @param, @return, @throws on every public function. Where a parameter has bounds, document them ("range 1..1000").
- Use @sample to point at a real test that demonstrates usage; Dokka will inline the snippet.
- Use @since on every public element so consumers can see when an API became available.
- Cross-link with [Class] and [Class.method] rather than free text.
- module.md describes the library at the package level; it is the home page of the generated site.

9.3 What gets published

Artifact	Audience	Path
HTML site	Public consumers, internal devs	build/docs/html (uploaded to a GitHub Pages site on tag)
Markdown	README links, internal wikis	build/docs/markdown
Sources jar	IDE quick-doc	Published to Maven alongside the AAR
Javadoc jar	Maven Central requirement	Generated via dokkaJavadoc

10. Configuration Examples

10.1 Minimal setup

```
class App : Application() {

override fun onCreate() {

super.onCreate()

EventTracker.initialize(

this,

EventTrackerConfig.Builder()

.addAdapter(BackendBatchAdapter("https://api.example.com/v1/events"))

.build()

)

}

}
```

10.2 Multi-destination with sampling and encryption

```
EventTracker.initialize(

this,

EventTrackerConfig.Builder()
```

```
.addAdapter(BackendBatchAdapter("https://api.example.com/v1/events"))
.addAdapter(FirebaseAdapter())
.addAdapter(MoEngageAdapter(appId = BuildConfig.MOE_APP_ID))
.batchSize(75)
.batchIntervalMs(20_000)
.maxRetries(10)
.maxLocalEvents(20_000)
.samplingRate("scroll", 0.05) // keep 5% of scroll events
.samplingRate("debug_breadcrumb", 0.0) // drop entirely in prod
.encryptAtRest(true)
.logger(TimberEventLogger())
.build()
)
```

10.3 Routing PII to the first-party backend only

```
EventTracker.track(
name = "payment_card_added",
properties = mapOf("last4" to "1234"),
destinations = setOf("backend") // explicitly NOT firebase / moengage
)
```

10.4 Operational call: replay DLQ after a backend fix

```
lifecycleScope.launch {
val replayed = EventTracker.replayDeadLetters(limit = 5_000)
Log.i("Ops", "Re-queued $replayed events for delivery")
EventTracker.flush().join()
}
```

11. Open Questions and Future Work

- Should partial-success (HTTP 207) be a v1 feature or v1.1? Backend team must commit either way.
- Compression algorithm — gzip is universal, but zstd would save ~20% bytes if backend supports it.
- Cross-platform variant — should we extract the core into a Kotlin Multiplatform module so iOS can share it?
- Foreground-only mode for low-end devices to avoid even WorkManager overhead.

- Debug overlay artifact (-debug variant) showing live diagnostics on a floating window.

12. Approval

Role	Name	Date	Signature
Android lead			
Backend lead			
Security review			
Product owner			